

Ejemplo de plan técnico para comenzar a construir el sistema

1. El recorrido desde la idea hasta una funcionalidad terminada

Cada tarea debería recorrer esta cadena:

```
Necesidad de negocio
→ reglas y decisiones pendientes
→ modelo de datos
→ migración
→ servicio transaccional
→ pruebas del servicio
→ selector de consultas
→ formulario
→ vista y URL
→ interfaz
→ permisos
→ auditoría
→ prueba integral
→ aceptación
```

Esto evita construir pantallas que funcionan visualmente, pero que permiten operaciones financieras inconsistentes.

2. Qué es un corte vertical

Un corte vertical es una función pequeña que atraviesa todas las capas del sistema.

Por ejemplo:

```
"Crear una rendición en borrador"
```

No significa solamente crear un formulario. Incluye:

```
Modelo y migración
Servicio para crearla
Numeración segura
Permisos
Formulario
Vista
URL
```

Template
Auditoría
Pruebas

Al terminar ese corte, un usuario puede crear una rendición real y el sistema garantiza que queda correctamente registrada.

3. Orden recomendado para construir Rendiciones

Como ya estás trabajando en este módulo, un plan razonable sería el siguiente.

Iteración 0 — Fundaciones necesarias

Antes de continuar con las funciones financieras:

Trabajo	Resultado
Separar <code>models.py</code> si ya es demasiado grande	Código mantenible
Consolidar <code>ModeloAuditoria</code>	Fechas y usuarios comunes
Crear servicios y selectores	Reglas separadas de vistas
Configurar grupos y permisos	Control de acceso
Definir adjuntos protegidos	Archivos seguros
Definir eventos de auditoría	Historial transversal
Crear factories de prueba	Datos de prueba repetibles

Estructura inicial:

```
administracion/  
├── models/  
│   ├── base.py  
│   ├── bancos.py  
│   ├── facturacion.py  
│   ├── rendiciones.py  
│   └── factoring.py  
├── services/  
├── selectors/  
├── forms/  
├── views/  
├── templates/  
└── tests/
```

No es obligatorio dividir inmediatamente todos los modelos, pero sí conviene establecer esta arquitectura antes de que el módulo crezca demasiado.

Iteración 1 — Maestros y cabecera

Tareas:

R003 Responsables
R004 Categorías
R002 Crear y editar rendición
R001 Lista y búsqueda

Resultado visible:

Usuario entra al módulo
→ crea un responsable
→ configura categorías
→ crea una rendición
→ la encuentra en el listado

Criterio para avanzar:

- Numeración única.
 - Responsable válido.
 - Período válido.
 - Borrador editable.
 - Permisos funcionando.
 - Pruebas aprobadas.
-

Iteración 2 — Fondos, gastos y documentos

Tareas:

R005 Entregas de fondo
R006 Gastos
R007 Adjuntos
R008 Validación documental
R009 Cálculos

Resultado visible:

Rendición
→ recibe fondos

- incorpora gastos
- adjunta comprobantes
- muestra errores
- calcula diferencia

Criterio para avanzar:

- Los fondos anulados no suman.
- Los gastos anulados no suman.
- Los montos usan **Decimal**.
- Los documentos están protegidos.
- Los duplicados se detectan.
- Los totales de listado y detalle coinciden.

Iteración 3 — Workflow

Tareas:

R010 Presentación
R011 Revisión
R012 Aprobación
R013 Historial

Resultado visible:

BORRADOR
→ PRESENTADA
→ EN REVISIÓN
→ OBSERVADA o APROBADA

Criterio para avanzar:

- No se edita después de presentar.
- El responsable no aprueba su propia rendición.
- Las observaciones no sobrescriben el historial.
- Las aprobaciones parciales quedan justificadas.
- Las transiciones inválidas son rechazadas.

Iteración 4 — Liquidación y banco

Tareas:

R014 Liquidación
R015 Devolución

R016 Reembolso
R017 Conciliación

Resultado visible:

Rendición aprobada
→ liquidación
└─ cuadrada
└─ responsable devuelve
└─ empresa reembolsa
→ conciliación bancaria
→ cierre

Criterio para avanzar:

- La liquidación conserva snapshot.
- No se paga más que la diferencia.
- Una devolución exige ingreso bancario.
- Un reembolso exige egreso bancario.
- No existen saldos negativos.
- El cierre solo ocurre al completar la regularización.

Iteración 5 — Operación y control

Tareas:

R018 Alertas
R019 Exportaciones
R020 Dashboard
R021 Pruebas integrales
R022 Manual

Resultado visible:

Módulo completo
→ alertas
→ dashboard
→ PDF y Excel
→ pruebas
→ manual

4. Ejemplo completo: implementar R014 — Liquidar una rendición

Este ejemplo muestra cómo una mini especificación se convierte en código.

`LiquidacionRendicion` ya existe en tu modelo y se relaciona uno a uno con `Rendicion`; también conserva fondos, gastos aprobados, diferencia y resultado.

Paso 1 — Escribir la regla de negocio en lenguaje simple

Una rendición solo puede liquidarse si está aprobada.

Al liquidar:

1. Se recalculan los fondos entregados.
2. Se recalculan los gastos aprobados.
3. Se calcula la diferencia.
4. Se determina quién debe pagar.
5. Se guarda un snapshot.
6. La rendición cambia a LIQUIDADA.
7. Se registra la acción.

Fórmula:

```
diferencia = fondos entregados - gastos aprobados
```

Interpretación:

```
diferencia = 0  
→ CUADRADA  
  
diferencia > 0  
→ RESPONSABLE_DEVUELVE  
  
diferencia < 0  
→ EMPRESA_REEMBOLSA
```

Paso 2 — Registrar decisiones abiertas

Antes de programar, se crea un documento pequeño:

ADR-014 – Liquidación de rendiciones

Un ADR es un registro de decisión arquitectónica.

Ejemplo:

```
Decisión 1:  
Los totales se recalcularán al liquidar.  
  
Decisión 2:  
Los valores definitivos se guardarán como snapshot.  
  
Decisión 3:  
Una liquidación no se editará directamente.  
  
Decisión 4:  
Las correcciones posteriores exigirán reapertura.  
  
Decisión 5:  
Los pagos parciales serán permitidos.
```

Esto evita que cada programador interprete las reglas de una manera diferente.

Paso 3 — Revisar el modelo

Una versión simplificada podría quedar así:

```
from decimal import Decimal  
  
from django.conf import settings  
from django.db import models  
  
class LiquidacionRendicion(ModeloAuditoria):  
    class Resultado(models.TextChoices):  
        CUADRADA = "CUADRADA", "Cuadrada"  
        RESPONSABLE_DEVUELVE = (  
            "RESPONSABLE_DEVUELVE",  
            "Responsable debe devolver",  
        )  
        EMPRESA_REEMBOLSA = (  
            "EMPRESA_REEMBOLSA",  
            "Empresa debe reembolsar",  
        )  
  
    class Estado(models.TextChoices):
```

```

PENDIENTE = "PENDIENTE", "Pendiente"
PARCIAL = "PARCIAL", "Parcial"
REGULARIZADA = "REGULARIZADA", "Regularizada"
CONCILIADA = "CONCILIADA", "Conciliada"
ANULADA = "ANULADA", "Anulada"

rendicion = models.OneToOneField(
    "Rendicion",
    on_delete=models.PROTECT,
    related_name="liquidacion",
)

total_fondos = models.DecimalField(
    max_digits=14,
    decimal_places=2,
)

total_declarado = models.DecimalField(
    max_digits=14,
    decimal_places=2,
)

total_aprobado = models.DecimalField(
    max_digits=14,
    decimal_places=2,
)

diferencia = models.DecimalField(
    max_digits=14,
    decimal_places=2,
)

resultado = models.CharField(
    max_length=30,
    choices=Resultado.choices,
)

estado = models.CharField(
    max_length=20,
    choices=Estado.choices,
    default=Estado.PENDIENTE,
)

datos_snapshot = models.JSONField(
    default=dict,
    blank=True,
)

liquidada_por = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.PROTECT,

```



```
        related_name="liquidaciones_realizadas",  
    )
```

Paso 4 — Crear la migración

```
python manage.py makemigrations administracion  
python manage.py migrate
```

Antes de aplicarla en producción:

```
python manage.py migrate --plan
```

La migración debe revisarse manualmente para comprobar:

- Que no elimine datos.
- Que no cambie nombres inesperadamente.
- Que los valores predeterminados sean correctos.
- Que las restricciones sean compatibles con registros existentes.

Paso 5 — Crear el servicio financiero

Este es el corazón de la funcionalidad.

```
# administracion/services/rendiciones/liquidacion.py  
  
from decimal import Decimal  
  
from django.core.exceptions import ValidationError  
from django.db import transaction  
from django.db.models import Sum  
from django.utils import timezone  
  
from administracion.models import (  
    LiquidacionRendicion,  
    Rendicion,  
)  
  
@transaction.atomic  
def liquidar_rendicion(  
    *,  
    rendicion_id: int,  
    usuario,  
    observaciones: str = "",
```

```

) -> LiquidacionRendicion:
    rendicion = (
        Rendicion.objects
        .select_for_update()
        .select_related("responsable")
        .get(pk=rendicion_id)
    )

    if rendicion.estado != Rendicion.Estado.APROBADA:
        raise ValidationError(
            "Solo se puede liquidar una rendición aprobada."
        )

    if hasattr(rendicion, "liquidacion"):
        raise ValidationError(
            "La rendición ya posee una liquidación."
        )

    total_fondos = (
        rendicion.entregas_fondo
        .filter(anulada=False)
        .aggregate(total=Sum("monto"))["total"]
        or Decimal("0.00")
    )

    detalles = rendicion.detalles.filter(
        anulado=False
    )

    total_declarado = (
        detalles.aggregate(total=Sum("total"))["total"]
        or Decimal("0.00")
    )

    total_aprobado = (
        detalles.filter(
            estado_revision="APROBADO"
        ).aggregate(
            total=Sum("monto_aprobado")
        )["total"]
        or Decimal("0.00")
    )

    diferencia = total_fondos - total_aprobado

    if diferencia == Decimal("0.00"):
        resultado = (
            LiquidacionRendicion
            .Resultado
            .CUADRADA
        )

```

```

        estado = (
            LiquidacionRendicion
            .Estado
            .REGULARIZADA
        )

elif diferencia > Decimal("0.00"):
    resultado = (
        LiquidacionRendicion
        .Resultado
        .RESPONSABLE_DEVUELVE
    )
    estado = (
        LiquidacionRendicion
        .Estado
        .PENDIENTE
    )

else:
    resultado = (
        LiquidacionRendicion
        .Resultado
        .EMPRESA_REEMBOLSA
    )
    estado = (
        LiquidacionRendicion
        .Estado
        .PENDIENTE
    )

snapshot = {
    "rendicion_numero": rendicion.numero,
    "responsable_id": rendicion.responsable_id,
    "responsable_nombre": (
        rendicion.responsable_nombre
    ),
    "total_fondos": str(total_fondos),
    "total_declarado": str(total_declarado),
    "total_aprobado": str(total_aprobado),
    "diferencia": str(diferencia),
    "generado_en": timezone.now().isoformat(),
    "observaciones": observaciones,
}

liquidacion = LiquidacionRendicion.objects.create(
    rendicion=rendicion,
    total_fondos=total_fondos,
    total_declarado=total_declarado,
    total_aprobado=total_aprobado,
    diferencia=diferencia,
    resultado=resultado,

```

```

        estado=estado,
        datos_snapshot=snapshot,
        liquidada_por=usuario,
        creado_por=usuario,
    )

    rendicion.estado = Rendicion.Estado.LIQUIDADA
    rendicion.fecha_liquidacion = timezone.now()
    rendicion.actualizado_por = usuario
    rendicion.save(
        update_fields=[
            "estado",
            "fecha_liquidacion",
            "actualizado_por",
            "actualizado_en",
        ]
    )

    return liquidacion

```

Puntos importantes:

```
transaction.atomic()
```

garantiza que todos los cambios se confirmen juntos.

```
select_for_update()
```

bloquea la rendición mientras se liquida, evitando que dos usuarios generen dos liquidaciones simultáneas.

Paso 6 — Crear las primeras pruebas antes de la pantalla

```

import pytest
from decimal import Decimal

from django.core.exceptions import ValidationError

from administracion.models import LiquidacionRendicion
from administracion.services.rendiciones.liquidacion import (
    liquidar_rendicion,
)

@pytest.mark.django_db
def test_liquidacion_cuadrada(

```

```

    rendicion_aprobada_factory,
    usuario_factory,
):
    usuario = usuario_factory()

    rendicion = rendicion_aprobada_factory(
        total_fondos=Decimal("500000.00"),
        total_aprobado=Decimal("500000.00"),
    )

    liquidacion = liquidar_rendicion(
        rendicion_id=rendicion.pk,
        usuario=usuario,
    )

    assert (
        liquidacion.resultado
        == LiquidacionRendicion.Resultado.CUADRADA
    )
    assert liquidacion.diferencia == Decimal("0.00")

@pytest.mark.django_db
def test_no_liquidar_borrador(
    rendicion_borrador_factory,
    usuario_factory,
):
    rendicion = rendicion_borrador_factory()
    usuario = usuario_factory()

    with pytest.raises(ValidationError):
        liquidar_rendicion(
            rendicion_id=rendicion.pk,
            usuario=usuario,
        )

```

Después se agregan:

```

Responsable devuelve
Empresa reembolsa
Liquidación duplicada
Fondos anulados
Gastos rechazados
Aprobación parcial
Concurrencia
Rollback ante error
Permisos

```

Paso 7 — Crear el formulario de confirmación

El usuario no debe ingresar:

- Total de fondos.
- Total aprobado.
- Diferencia.
- Resultado.

Esos valores los calcula el backend.

```
from django import forms

class LiquidarRendicionForm(forms.Form):
    observaciones = forms.CharField(
        required=False,
        widget=forms.Textarea(
            attrs={"rows": 4}
        ),
    )

    confirmar = forms.BooleanField(
        required=True,
        label=(
            "Confirmo que los montos y decisiones "
            "de revisión fueron verificados."
        ),
    )
```

Paso 8 — Crear la vista

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    PermissionRequiredMixin,
)
from django.shortcuts import get_object_or_404, redirect
from django.views.generic import FormView

from administracion.models import Rendicion
from administracion.services.rendiciones.liquidacion import (
    liquidar_rendicion,
)

class RendicionLiquidarView(
    LoginRequiredMixin,
```

```

PermissionRequiredMixin,
FormView,
):
    template_name = (
        "administracion/rendiciones/"
        "liquidacion/confirmar.html"
    )
    form_class = LiquidarRendicionForm
    permission_required = (
        "administracion.liquidar_rendicion"
    )

    def dispatch(self, request, *args, **kwargs):
        self.rendicion = get_object_or_404(
            Rendicion,
            pk=kwargs["pk"],
        )
        return super().dispatch(
            request,
            *args,
            **kwargs,
        )

    def form_valid(self, form):
        liquidacion = liquidar_rendicion(
            rendicion_id=self.rendicion.pk,
            usuario=self.request.user,
            observaciones=(
                form.cleaned_data["observaciones"]
            ),
        )

        return redirect(
            "administracion:liquidacion_detail",
            pk=liquidacion.pk,
        )

```

La vista no calcula montos. Solo:

- Valida acceso.
- Lee el formulario.
- Llama al servicio.
- Redirige.

Paso 9 — Crear la URL

```

path(
    "rendiciones/<int:pk>/liquidar/",

```

```
RendicionLiquidarView.as_view(),
name="rendicion_liquidar",
),
```

Paso 10 — Crear la pantalla

```
<h1>Liquidar rendición {{ rendicion.numero }}</h1>

<table>
  <tr>
    <th>Fondos entregados</th>
    <td>{{ resumen.total_fondos }}</td>
  </tr>
  <tr>
    <th>Gastos declarados</th>
    <td>{{ resumen.total_declarado }}</td>
  </tr>
  <tr>
    <th>Gastos aprobados</th>
    <td>{{ resumen.total_aprobado }}</td>
  </tr>
  <tr>
    <th>Diferencia preliminar</th>
    <td>{{ resumen.diferencia }}</td>
  </tr>
</table>

<form method="post">
  {% csrf_token %}
  {{ form.as_p }}

  <button type="submit">
    Confirmar liquidación
  </button>
</form>
```

El resumen debería venir desde un selector, no calcularse dentro del template.

Paso 11 — Crear el selector

```
# selectors/rendiciones.py

from decimal import Decimal

from django.db.models import Sum
```



```

def obtener_resumen_liquidacion(rendicion):
    total_fondos = (
        rendicion.entregas_fondo
        .filter(anulada=False)
        .aggregate(total=Sum("monto"))["total"]
        or Decimal("0.00")
    )

    total_declarado = (
        rendicion.detalles
        .filter(anulado=False)
        .aggregate(total=Sum("total"))["total"]
        or Decimal("0.00")
    )

    total_aprobado = (
        rendicion.detalles
        .filter(
            anulado=False,
            estado_revision="APROBADO",
        )
        .aggregate(
            total=Sum("monto_aprobado")
        )["total"]
        or Decimal("0.00")
    )

    return {
        "total_fondos": total_fondos,
        "total_declarado": total_declarado,
        "total_aprobado": total_aprobado,
        "diferencia": (
            total_fondos - total_aprobado
        ),
    }

```

El selector consulta y presenta datos.

El servicio cambia el estado del sistema.

Esta separación es importante:

Selector

→ lectura

Servicio

→ escritura y reglas

Paso 12 — Registrar auditoría

Después de crear la liquidación se registra un evento:

```
registrar_evento_auditoria(  
  tipo="RENDICION_LIQUIDADA",  
  objeto=liquidacion,  
  usuario=usuario,  
  datos={  
    "rendicion": rendicion.numero,  
    "total_fondos": str(total_fondos),  
    "total_aprobado": str(total_aprobado),  
    "diferencia": str(diferencia),  
    "resultado": resultado,  
  },  
)
```

5. Cómo convertir la funcionalidad en tickets

La tarea R014 puede transformarse así:

Ticket	Trabajo	Dependencia
R014-T01	Revisar reglas y decisiones	Ninguna
R014-T02	Ajustar modelo de liquidación	T01
R014-T03	Crear migración	T02
R014-T04	Crear selector de resumen	T02
R014-T05	Crear servicio transaccional	T03
R014-T06	Pruebas unitarias del servicio	T05
R014-T07	Formulario de confirmación	T04
R014-T08	Vista y URL	T05 y T07
R014-T09	Template	T08
R014-T10	Permisos y auditoría	T05
R014-T11	Prueba de integración	T08-T10
R014-T12	Revisión funcional	Todos

Así una tarea de 8 horas deja de ser una instrucción genérica y se convierte en trabajo verificable.

6. Plantilla para cualquier ticket técnico

Código:

R014-T05

Título:

Crear servicio de liquidación

Objetivo:

Crear una liquidación consistente a partir de una rendición aprobada.

Entrada:

- rendicion_id
- usuario
- observaciones

Reglas:

- Solo estado APROBADA.
- No debe existir otra liquidación.
- Recalcular fondos y aprobados.
- Determinar resultado automáticamente.
- Guardar snapshot.
- Cambiar a LIQUIDADA.

Archivos:

- services/rendiciones/liquidacion.py
- tests/rendiciones/test_liquidacion.py

Pruebas:

- cuadrada
- devolución
- reembolso
- estado inválido
- duplicada
- concurrencia

Criterio de terminado:

Todas las pruebas pasan y la operación queda auditada.

7. Cómo abordar lo que todavía no dominas

No es necesario dominar cada aspecto antes de comenzar. Se puede trabajar con cuatro mecanismos.

7.1 Spike técnico

Un spike es una investigación corta antes de implementar algo incierto.

Ejemplos:

SPIKE-01

¿Cómo proteger archivos media en Django?

SPIKE-02

¿Cómo probar `select_for_update`?

SPIKE-03

¿Cómo generar Excel sin fórmulas peligrosas?

SPIKE-04

¿Cómo visualizar PDF sin exponer su URL?

El resultado del spike no es una función productiva. Es:

- Una conclusión.
- Un ejemplo pequeño.
- Una decisión.
- Riesgos identificados.

7.2 Prototipo desechable

Para una pantalla compleja, puede construirse primero:

HTML sin reglas financieras

Solo para validar:

- Distribución.
- Campos.
- Navegación.
- Comprensión del usuario.

Después se conecta al servicio real.

El prototipo no debe convertirse accidentalmente en código productivo sin validaciones.

7.3 Prueba antes de la función

Cuando una regla sea delicada, se escribe primero el caso.

Ejemplo:

```
def test_no_se_puede_reembolsar_mas_que_la_diferencia():  
    ...
```

Después se escribe el servicio hasta que la prueba pase.

Esto ayuda especialmente en:

- Aplicaciones parciales.
- Conciliaciones.
- Factoring.
- Devoluciones.
- Reversas.
- Concurrencia.

7.4 Registro de decisiones

Crear:

```
docs/decisiones/
```

Ejemplos:

```
ADR-001 – Los montos usan Decimal  
ADR-002 – Las aplicaciones no se eliminan  
ADR-003 – Las facturas cedidas no reciben pagos normales  
ADR-004 – Los cierres conservan snapshot  
ADR-005 – Los archivos se descargan mediante vista protegida
```

Esto evita olvidar por qué se construyó una función de determinada manera.

8. Cómo hacer visible el progreso

Cada iteración debería terminar con una demostración pequeña.

Demostración 1

```
Crear responsable y rendición.
```

Demostración 2

Agregar fondo, gasto y comprobante.

Demostración 3

Presentar, observar y corregir.

Demostración 4

Aprobar y liquidar.

Demostración 5

Conciliar devolución o reembolso.

Así podrás ver aparecer el sistema progresivamente y comprobar las reglas antes de construir todo el módulo.

9. Tablero de seguimiento recomendado

Columnas:

Por definir
Lista para desarrollar
En desarrollo
En revisión
En pruebas
En aceptación
Terminada
Bloqueada

Cada ticket debe tener:

- Módulo.
- Código Gantt.
- Prioridad.
- Dependencias.
- Archivos afectados.
- Pruebas.
- Responsable.

- Estado.

10. Definición de listo antes de programar

Una tarea está lista para desarrollo cuando:

- El objetivo es claro.
- Las reglas están escritas.
- Los estados están definidos.
- Las decisiones abiertas están resueltas.
- Los permisos están identificados.
- Los modelos involucrados están definidos.
- Los criterios de aceptación existen.
- Los datos de prueba están disponibles.

Esto se conoce como:

Definition of Ready

11. Definición de terminado

Una tarea no está terminada solamente porque aparece en pantalla.

Debe cumplir:

Código

- + migración
- + servicio
- + permisos
- + auditoría
- + pruebas
- + interfaz
- + documentación mínima

Checklist:

- La migración funciona.
- Las reglas están en un servicio.
- Las pruebas pasan.
- Los permisos se verifican en backend.
- Los errores muestran mensajes comprensibles.
- No existen consultas N+1 importantes.
- Los cambios quedan auditados.
- El caso fue demostrado.

- El usuario funcional lo comprende.

Esto se conoce como:

Definition of Done

12. Primer plan práctico de trabajo

Semana o iteración inicial

Día 1 — Arquitectura

- Crear carpetas.
- Revisar modelos actuales.
- Definir servicios y selectores.
- Crear documento de decisiones.
- Configurar pytest y factories.

Día 2 — Responsables y categorías

- CRUD.
- Validaciones.
- Permisos.
- Pruebas.

Día 3 — Cabecera de rendición

- Numeración.
- Período.
- Responsable.
- Borrador.
- Listado.

Día 4 — Fondo y gasto básico

- Entrega de fondo.
- Registro de gasto.
- Totales iniciales.
- Pruebas.

Día 5 — Demostración

Responsable
→ rendición
→ fondo
→ gasto
→ total

Durante esta primera iteración todavía no se aprueba ni liquida. Se construye un primer corte estable sobre el cual continuar.

13. Resultado esperado de esta forma de trabajo

Al aplicar este método, cada funcionalidad tendrá:

Una regla clara
Una ubicación clara
Un servicio responsable
Pruebas repetibles
Una interfaz comprobable
Un historial verificable

Y el proyecto avanzará así:

Especificación
→ ticket
→ prueba
→ servicio
→ interfaz
→ demostración
→ aceptación

No será necesario intentar comprender o construir el sistema completo de una sola vez. Cada iteración agregará una pieza terminada y protegida, manteniendo la consistencia de las piezas anteriores.